

TEXTBOOK EDITION • RAPID REVISION & EMERGENCY CRAMMING

Final 24–Hour High–Yield Revision & Flashcards Pack

Rapid Cheatsheets across All Disciplines & 35+ Active Recall Flashcards

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	4 Hours
DIFFICULTY LEVEL	High–Yield Summary
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — DSA Last-Minute Algorithmic Patterns Cheatsheet

Study Time: 35m

Chapter 02 — SQL & DBMS Last-Minute Interview Cheatsheet

Study Time: 35m

Chapter 03 — Node.js & Backend Architecture Last-Minute Cheatsheet

Study Time: 35m

Chapter 04 — System Design Last-Minute Interview Cheatsheet

Study Time: 35m

Chapter 05 — Last-Minute Python Data Ecosystem Cheatsheet

Study Time: 35m

Chapter 06 — 10-Minute Rapid Revision: Tricky JS, Python & SQL Gotchas

Study Time: 35m

Chapter 07 — Active Recall Flashcards for IDFC FIRST Bank Interview

Study Time: 35m

CHAPTER 01

DSA Last–Minute Algorithmic Patterns Cheatsheet

PRIORITY ● High Priority	DIFFICULTY High–Yield Summary	EST. TIME 35–45 min	FREQUENCY Very High (85%+)
--	---	---	--

DSA Last–Minute Algorithmic Patterns Cheatsheet

The Top 14 Algorithmic Patterns Quick Reference

Pattern	Trigger in Problem Statement	Core Data Structure	Typical Complexity
1. Frequency Hash Map	"Find duplicates", "Frequency of elements", "Anagrams"	<code>collections.Counter</code> / <code>dict</code>	O(N) Time, O(N) Space
2. Two Pointers	"Sorted array", "Find pair summing to target", "Palindromes"	Two integer indices (<code>L</code> , <code>R</code>)	O(N) Time, O(1) Space
3. Sliding Window	"Contiguous subarray/substring", "Longest/Shortest with condition"	Two indices + Hash Map	O(N) Time, O(K) Space
4. Fast & Slow Pointers	"Cycle detection in linked list", "Find middle of linked list"	<code>slow</code> , <code>fast</code> pointers	O(N) Time, O(1) Space
5. Prefix Sum	"Subarray sum equals K", "Range sum queries"	Array / Running sum	O(N) Time, O(1) query
6. Monotonic Stack	"Next greater element", "Daily temperatures", "Stock span"	<code>list</code> as stack	O(N) Time, O(N) Space
7. Top K Elements	"Find K largest / K most frequent", "Running median"	<code>heapq</code> (Min-Heap / Max-Heap)	O(N log K) Time
8. Binary Search	"Sorted input", "Find peak", "Search rotated sorted array", "Min of Max"	<code>left</code> , <code>right</code> , <code>mid</code>	O(log N) Time, O(1) Space
9. BFS (Level Order)	"Shortest path in unweighted graph/grid", "Level-by-level tree"	<code>collections.deque</code>	O(V + E) Time, O(V) Space
10. DFS (Backtracking)	"Generate all combinations/permutations/subsets", "Islands in grid"	Recursion / Call Stack	O(2 ^N) or O(N!) Time
11. Merge Intervals	"Overlapping intervals", "Meeting rooms schedule"	Sort by <code>start_time</code>	O(N log N) Time
12. Dynamic Programming	"Overlapping subproblems", "Max/Min path", "Number of ways"	1D / 2D table or memo dict	O(N) or O(N * M)
13. Greedy	"Interval scheduling", "Jump game", "Local optimum yields global"	Priority Queue or Sort	O(N log N) Time
14. Union-Find (DSU)	"Connected components", "Detect cycle in undirected graph"	<code>parent</code> and <code>rank</code> arrays	O(alpha(N)) ≈ O(1)

Python High-Yield Syntax Snippets

```
# 1. Monotonic Deque for O(1) pops and appends:
from collections import deque
q = deque()
q.append(1)
q.popleft() # O(1) vs list.pop(0) which is O(N)!

# 2. Min-Heap & Max-Heap:
import heapq
min_h = []
heapq.heappush(min_h, 10)
val = heapq.heappop(min_h)
# Max-heap: push negative values
heapq.heappush(min_h, -10)
val = -heapq.heappop(min_h)

# 3. Safe Binary Search Mid Calculation:
mid = left + (right - left) // 2

# 4. In-Place Reversal:
def reverse_range(arr, l, r):
    while l < r:
        arr[l], arr[r] = arr[r], arr[l]
        l += 1
        r -= 1

# 5. Defaultdict without KeyError:
from collections import defaultdict
graph = defaultdict(list)
counts = defaultdict(int)
```

CHAPTER 01 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of DSA Last-Minute Algorithmic Patterns Cheatsheet
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

SQL & DBMS Last-Minute Interview Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

High-Yield Summary

EST. TIME

35-45 min

FREQUENCY

Very High (85%+)

SQL & DBMS Last-Minute Interview Cheatsheet

 SQL Logical Execution Order

1. FROM & JOIN (Combines tables)
2. ON (Filters join rows)
3. WHERE (Filters base rows before grouping - NO AGGREGATES!)
4. GROUP BY (Aggregates rows into buckets)
5. HAVING (Filters aggregated groups)
6. SELECT (Calculates expressions & column aliases)
7. DISTINCT (Deduplicates output)
8. ORDER BY (Sorts results)
9. LIMIT / OFFSET (Paginates results)

 Window Functions Cheat Sheet

```
-- General Syntax:
FUNCTION() OVER (
  PARTITION BY category_col
  ORDER BY sort_col
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- Key Ranking Differences:
ROW_NUMBER() -- 1, 2, 3, 4 (Always unique sequential integers)
RANK()       -- 1, 2, 2, 4 (Ties get same rank, subsequent rank skipped)
DENSE_RANK() -- 1, 2, 2, 3 (Ties get same rank, NO rank skipped - Top N salaries!)

-- Navigation:
LAG(col, 1)  -- Reads value from previous row (Gap / time difference)
LEAD(col, 1) -- Reads value from next row

-- Running Total (Explicit Frame is faster than default RANGE!):
SUM(amount) OVER (
  PARTITION BY account_id
  ORDER BY created_at
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

ACID & Database Concurrency Cheat Sheet

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Default In
Read Uncommitted	✗ Allowed	✗ Allowed	✗ Allowed	—
Read Committed	✓ Prevented	✗ Allowed	✗ Allowed	Postgres / Oracle
Repeatable Read	✓ Prevented	✓ Prevented	✓ Prevented in Postgres	MySQL (InnoDB)
Serializable	✓ Prevented	✓ Prevented	✓ Prevented	SQLite / High Security

Locking Commands

```
-- Pessimistic Lock (Blocks other readers and writers):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE;

-- Pessimistic Lock (Fail immediately if locked):
SELECT * FROM accounts WHERE account_id = 101 FOR UPDATE NOWAIT;

-- Task Queue Processing (Skip locked rows to prevent worker pileup):
SELECT * FROM job_queue WHERE status = 'PENDING' LIMIT 1 FOR UPDATE SKIP LOCKED;

-- Optimistic Lock Check:
UPDATE accounts
SET balance = balance - 100, version = version + 1
WHERE account_id = 101 AND version = :read_version;
```

B+ Tree Indexing Rules

- Leftmost Prefix Rule:** If composite index is on (A, B, C) :
- WHERE A = 1 → **Index Scan (Yes)**
- WHERE A = 1 AND B = 2 → **Index Scan (Yes)**
- WHERE B = 2 AND C = 3 → **Sequential Scan (No index used!)**
- Range Breaks Indexing:** If query has WHERE A = 1 AND B > 5 AND C = 10 :
- Column A uses index seek.
- Column B uses index range scan.
- Column C **cannot** use the index; must be filtered after scan!
- Clustered Index:** Leaf nodes contain full data rows (1 per table).
- Secondary Index:** Leaf nodes contain indexed column + Primary Key pointer.
- Covering Index:** Index contains *all* projected columns, bypassing table data pages entirely.



CHAPTER 02 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- `WHERE A = 1` → Index Scan (Yes)
- `WHERE A = 1 AND B = 2` → Index Scan (Yes)
- `WHERE B = 2 AND C = 3` → Sequential Scan (No index used!)



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

Node.js & Backend Architecture Last-Minute Cheatsheet

PRIORITY

 High Priority

DIFFICULTY

High-Yield Summary

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Node.js & Backend Architecture Last-Minute Cheatsheet

libuv Event Loop Phase Order

Synchronous Call Stack (V8)



[Microtask Drain: process.nextTick → Promise Microtasks]


 1. Timers Phase (setTimeout, setInterval callbacks)
 [Microtask Drain]

 2. Pending I/O Phase (System/TCP error callbacks)
 [Microtask Drain]

 3. Idle / Prepare (libuv internal use)
 [Microtask Drain]

 4. Poll Phase (Retrieve I/O events via epoll/kqueue; executes I/O callbacks)
 [Microtask Drain]

 5. Check Phase (setImmediate callbacks)
 [Microtask Drain]

 6. Close Callbacks (socket.on('close'))
 [Microtask Drain]

Key Differences & Interview Gotchas

1. `process.nextTick()` vs `setImmediate()` :
2. `process.nextTick()` is **not** part of libuv; executes immediately after current synchronous code. Infinite recursion freezes the server!
3. `setImmediate()` runs during the libuv **Check phase**.
4. Inside an I/O callback, `setImmediate` **always runs before** `setTimeout(0)` .
5. **What uses the libuv Worker Thread Pool (`UV_THREADPOOL_SIZE = 4`)?**
6. **Uses Thread Pool:** `fs` (file system), `crypto` (`pbkdf2` , `scrypt`), `zlib` (compression), `dns.lookup` .

7. **Does NOT use Thread Pool:** Network sockets (`http` , `https` , `tls` , `net`), `dns.resolve` (handled non-blocking via OS `epoll` / `kqueue`).

8. **Streams & Backpressure:** ``javascript // Anti-Pattern (Leaks RAM when client network is slow):
fs.readFile('huge.csv', (err, data) => res.send(data));

// Production Pattern with Backpressure (Stream pipeline): const { pipeline } = require('stream/promises'); await pipeline(fs.createReadStream('huge.csv'), res); ``

Memory Leak Detection in Node.js

- **Tool:** Chrome DevTools (`node --inspect`) or `v8.writeHeapSnapshot()` .
- **Top 3 Culprits:**
 - Unbounded global caching objects without TTL/eviction.
 - Unremoved event listeners (`emitter.on` inside request handlers).
 - Stale closures retaining large outer function scope objects.
- **Metric to watch:** `process.memoryUsage().heapUsed` vs `heapTotal` .

Clustering vs Worker Threads

Dimension	Cluster Module (<code>cluster</code>)	Worker Threads (<code>worker_threads</code>)
Model	Multi-Process (Forking)	Multi-Thread within single process
Memory	Isolated memory space per core	Shared memory via <code>SharedArrayBuffer</code>
Port Sharing	Shares same server port via master proxy	Run within existing master process
Best Used For	Scaling web servers across CPU cores	Heavy CPU tasks (image resize, crypto)

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- `process.nextTick()` is not part of libuv; executes immediately after current synchronous code. Infinite recursion freez
- `setImmediate()` runs during the libuv Check phase.
- Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)`.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

System Design Last-Minute Interview Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

High-Yield Summary

EST. TIME

35-45 min

FREQUENCY

Very High (85%+)

System Design Last-Minute Interview Cheatsheet

1234 Numbers Every Engineer Must Know (Latencies)

L1 Cache Reference:	~0.5 ns
Branch Mispredict:	~5 ns
L2 Cache Reference:	~7 ns
Mutex Lock/Unlock:	~25 ns
Main Memory (RAM) Reference:	~100 ns
Compress 1KB with Snappy:	~3 µs
Send 1KB over 1Gbps Network:	~10 µs
Read 1MB Sequentially from SSD:	~1 ms
Disk Seek (HDD):	~10 ms
Send Packet Across Datacenter:	~0.5 ms
Round Trip (Mumbai to US-East):	~150 ms

High Availability & Uptime SLAs

Availability Target	Annual Allowed Downtime	Daily Allowed Downtime	Typical Tier
99% ("Two Nines")	3.65 days	14.4 minutes	Internal testing tools
99.9% ("Three Nines")	8.76 hours	43.2 seconds	Standard SaaS apps
99.99% ("Four Nines")	52.6 minutes	4.32 seconds	High-priority API gateways
99.999% ("Five Nines")	5.26 minutes	0.43 seconds	Core Banking Solution & Payment Rails

Redis Caching Strategies & Defenses

1. **Cache-Aside (Lazy Loading):** App reads cache. On miss, reads DB, writes to cache, returns. (Most common).

2. **Write-Through:** App writes to cache; cache synchronously writes to DB before acknowledging.
3. **Write-Behind (Write-Back):** App writes to cache; cache asynchronously batches writes to DB. (High write throughput, risk of data loss on crash).
4. **Cache Penetration Defense:** Querying keys that don't exist in DB.
5. *Fix:* Cache `null` with short TTL, or use a **Bloom Filter** at the gateway.
6. **Cache Stampede / Breakdown:** Hot key with high QPS expires; thousands of requests hit DB simultaneously.
7. *Fix:* Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).

Apache Kafka Quick Architecture

- **Topic:** Logical stream of messages.
- **Partition:** Ordered, append-only log of immutable messages. The unit of parallelism.
- **Partition Key:** Messages with the same key (e.g., `account_id`) **always go to the same partition**, guaranteeing chronological ordering per account.
- **Consumer Group:** Multiple workers reading a topic. Each partition is consumed by strictly **one** consumer within a group.
- **Replication Factor:** N copies across brokers (typical in banking: 3 replicas with `min.insync.replicas = 2` and `acks = all` for zero message loss).

CHAPTER 04 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- **Fix:** Cache `null` with short TTL, or use a Bloom Filter at the gateway.
- **Fix:** Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).
- Topic: Logical stream of messages.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

Last-Minute Python Data Ecosystem Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

High-Yield Summary

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

⚡ Last-Minute Python Data Ecosystem Cheatsheet

🐍 Python in 5 Minutes

1. Types & Mutability

- **Immutable (Cannot change in-place):** `int`, `float`, `str`, `tuple`, `frozenset`, `bytes`.
- **Mutable (In-place changes):** `list`, `dict`, `set`, `bytearray`.
- **List vs Tuple:** List mutable hai (`[1, 2]`); Tuple immutable hai (`(1, 2)`) - faster, hashable, can be dict keys).
- **Dict vs Set:** Dict key-value map hai; Set unique keys only hai (both $O(1)$ avg lookups).

2. Copying & References

- `b = a` $\Rightarrow$ Same pointer (mutating `b` mutates `a`).
- `shallow = a.copy()` $\Rightarrow$ New outer container, shared nested objects.
- `deep = copy.deepcopy(a)` $\Rightarrow$ Completely independent recursive clone.

3. Comprehensions & Packing

- `[x*2 for x in arr if x > 0]` (List) | `{k: v for k, v in pairs}` (Dict)
- `def f(*args, **kwargs):` `*args` captures tuple, `**kwargs` captures dictionary.

4. Generators, Decorators & Context Managers

- **Generator** (`yield`): Lazy evaluation, produces 1 item at a time in $O(1)$ memory.
- **Decorator** (`@func`): Wraps another function to add logging/auth without altering code.
- **Context Manager** (`with`): Guarantees `__exit__()` cleanup on DB connections/files.

1 2 3 4 NumPy in 3 Minutes

1. ndarray Anatomy

- **Contiguous memory:** C-style contiguous buffer of homogeneous bytes (unlike scattered Python lists).

- **shape** : Dimension tuple (e.g., `(2, 3)` = 2 rows, 3 cols).
- **axis=0** : Down rows (column-wise). **axis=1** : Across cols (row-wise).
- **dtype** : Data type (`int64`, `float32`).

2. Slicing & Reshape

- **Slices are Views**: Modifying `arr[1:3]` modifies the original array! Use `.copy()` if you need independence.
- **reshape(3, -1)** : `-1` auto-calculates remaining dimension.
- **flatten()** : Always returns a copy. **ravel()** : Returns a view when possible.

3. Broadcasting & Vectorization

- **Broadcasting Rules**: Dimensions match or one of them is 1 (stretches along size 1).
- **Vectorization**: Replaces slow Python loops with hardware SIMD C-level CPU instructions (40x–80x faster).



pandas in 5 Minutes

1. Core Structures

- **Series**: 1D labeled array. **DataFrame**: 2D labeled table of Series.

2. Selection: `loc` vs `iloc`

- **df.loc['row', 'col']** : **Label-based** (inclusive of end boundary).
- **df.iloc[0:5, 0:2]** : **Integer-position based** (0-indexed, exclusive of end boundary).

3. Filtering & Cleaning

- Multiple filters: `df[(df['amt'] > 1000) & (df['status'] == 'SUCCESS')]` (use `&` and `|` with parentheses).
- Missing Data: `df.isna().sum()` | `df.dropna()` | `df['amt'].fillna(df['amt'].median())`.
- Duplicates: `df.drop_duplicates(subset=['txn_id'])`.

4. GroupBy & Merging

- **Split-Apply-Combine**: `python df.groupby('category')['amt'].agg(['sum', 'mean', 'count']).reset_index()`
- **SQL Merges**: `python pd.merge(df1, df2, on='cust_id', how='left')` # SQL: LEFT JOIN
`pd.concat([df1, df2], axis=0)` # SQL: UNION ALL
- **Avoid `iterrows()`** : Vectorize with `np.select` or list comprehensions instead of loops.



Matplotlib in 2 Minutes

1. Anatomy: Figure vs Axes

- **Figure:** The overall canvas/window.
- **Axes:** The actual individual plot/graph (x-axis, y-axis, data points, title).

2. Object–Oriented Interface (Recommended)

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, color="#9b1c1c", label="Trend")
ax.set_title("IDFC Transaction Volume")
ax.set_xlabel("Date")
ax.set_ylabel("Volume (₹)")
ax.grid(True, linestyle="--")
ax.legend()
plt.tight_layout()
fig.savefig("chart.png", dpi=300)
plt.close(fig)
```

3. Chart Types

- **Line** (`ax.plot`): Time-series trends.
- **Bar** (`ax.bar`): Categorical comparisons.
- **Histogram** (`ax.hist`): Numerical distribution.
- **Scatter** (`ax.scatter`): Relationship / correlations between two variables.



CHAPTER 05 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Immutable (Cannot change in-place): ``int`, `float`, `str`, `tuple`, `frozenset`, `bytes``.
- Mutable (In-place changes): ``list`, `dict`, `set`, `bytearray``.
- List vs Tuple: List mutable hai (``[1, 2]``); Tuple immutable hai (``(1, 2)`` - faster, hashable, can be dict keys).



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 06

10–Minute Rapid Revision: Tricky JS, Python & SQL Gotchas

PRIORITY

● High Priority

DIFFICULTY

High–Yield Summary

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

10–Minute Rapid Revision: Tricky JS, Python & SQL Gotchas

The High–Yield Cram Sheet for IDFC FIRST Bank Technical Rounds

JavaScript Tricky Rapid Fire

Topic	The Code / Trap	Correct Answer & Why
Microtask Priority	<code>setTimeout(f, 0)</code> vs <code>Promise.resolve().then(f)</code>	<code>Promise</code> runs first . Call stack empties -> Microtasks completely drained -> Macrotasks execute.
<code>var</code> in Loop	<code>for (var i=0; i<3; i++) setTimeout(()=>console.log(i), 0)</code>	Prints <code>3, 3, 3</code> . <code>var</code> is function-scoped; all closures share the same <code>i</code> . Fix: use <code>let</code> .
Currency Math	<code>0.1 + 0.2 === 0.3</code>	Evaluates to <code>false</code> (<code>0.30000000000000004</code>) due to IEEE 754 floating point. In banking, always compute in paise / cents (integers) or <code>BigInt</code> .
Temporal Dead Zone	Accessing <code>let x</code> before declaration	Throws <code>ReferenceError</code> . <code>let</code> and <code>const</code> are hoisted into TDZ without initialization.
<code>this</code> in Arrow Func	<code>const obj = { a: 10, b: () => this.a }</code>	<code>obj.b()</code> returns <code>undefined</code> . Arrow functions inherit <code>this</code> from the enclosing lexical scope (global/module), not <code>{}</code> .
Type Coercion	<code>[] == ![]</code>	Evaluates to <code>true</code> ! <code>![] -> false -> 0</code> ; <code>[] -> "" -> 0</code> ; <code>0 == 0</code> is <code>true</code> .
Prototype Pollution	<code>const map = Object.create(null)</code>	Has NO prototype chain (<code>__proto__ === undefined</code>), preventing inherited properties and prototype pollution.
Debounce vs Throttle	Search autocomplete vs Payment submit button	Debounce : Waits for quiet period before running (search). Throttle : Guarantees execution at most once per interval (payment button).
Promise Concurrency	<code>Promise.all</code> vs <code>Promise.allSettled</code>	<code>all</code> fails-fast on the first rejection. <code>allSettled</code> waits for all to complete regardless of rejection.
V8 Hidden Classes	Adding properties in different order to objects	De-optimizes V8 into slow dictionary mode! Always initialize properties in the exact same order in constructors.

Python Tricky Rapid Fire

Topic	The Code / Trap	Correct Answer & Why
Small Int Cache	<code>a = 256; b = 256; a is b</code> vs <code>257</code>	<code>256</code> is <code>True</code> (cached from <code>-5</code> to <code>256</code>). <code>257</code> is <code>False</code> in REPL (separate heap allocations). Always use <code>=</code> for values.
Mutable Default Arg	<code>def f(val, acc=[]):</code> <code>acc.append(val); return</code> <code>acc</code>	Subsequent calls without 2nd arg share the same list in <code>f.__defaults__</code> ! Fix: <code>acc=None</code> .
Late-Binding Closure	<code>[lambda x: x*i for i in</code> <code>range(4)]</code>	All functions evaluate with <code>i=3</code> when called! Fix: bind default arg <code>lambda x, i=i: x*i</code> .
Tuple Mutation	<code>t = ([1, 2], 3); t[0] +=</code> <code>[4, 5]</code>	Raises <code>TypeError</code> AND mutates list! <code>+=</code> mutates list in-place via <code>__iadd__</code> , then fails when assigning back to tuple.
GIL & CPU Parallelism	Why <code>threading</code> doesn't speed up heavy math	CPython GIL allows only 1 native thread to execute bytecode at a time. Use <code>multiprocessing</code> or C-extensions (NumPy).
NumPy & GIL	Does NumPy array math hold the GIL?	No. NumPy releases the GIL during vectorized C/Fortran array computations across multiple cores.
C3 MRO & <code>super()</code>	Diamond inheritance <code>class D(B,</code> <code>C)</code>	<code>super()</code> calls the next class in the MRO list , NOT necessarily the direct parent! Resolves diamond without duplicates.
<code>__slots__</code> Optimization	<code>__slots__ = ('id', 'amt')</code>	Prevents per-instance <code>__dict__</code> creation. Reduces memory footprint by 40% to 60% in large ledgers.
<code>__exit__</code> Exception	Context manager <code>__exit__</code> returns <code>True</code>	Suppresses the exception and continues normal execution. Returning <code>False</code> re-raises.
<code>finally</code> Return Trap	<code>try: return 100 finally:</code> <code>return 200</code>	Returns <code>200</code> ! An explicit <code>return</code> in <code>finally</code> silently overwrites any return from <code>try</code> .

SQL & DBMS Tricky Rapid Fire

Topic	The Query / Trap	Correct Answer & Why
NOT IN with NULL	<code>WHERE id NOT IN (SELECT id FROM t)</code> where <code>t.id</code> has a <code>NULL</code>	Returns 0 rows ! <code>x <> NULL</code> is <code>UNKNOWN</code> , making the entire <code>AND</code> condition <code>UNKNOWN</code> . Always use <code>NOT EXISTS</code> .
Double-Spending	Concurrent UPI debits reading same balance	Use atomic in-place decrement <code>UPDATE accounts SET balance = balance - :amt WHERE id = :id AND balance ≥ :amt</code> or <code>SELECT ... FOR UPDATE</code> .
Deadlock Avoidance	Transferring between Account A and Account B	Deterministic lock ordering: Always lock accounts in sorted order by <code>account_id</code> (<code>first, second = sorted([a, b])</code>).
Running Total Frame	Default <code>OVER (ORDER BY dt)</code> vs explicit <code>ROWS</code>	Default uses <code>RANGE</code> , combining rows with duplicate timestamps. For true ledger step-by-step balance, use <code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code> .
Tied Ranks	Find 2nd highest balance	Use <code>DENSE_RANK() OVER (ORDER BY balance DESC)</code> . <code>LIMIT 1 OFFSET 1</code> fails if two top accounts have identical balances.
Anti-Join Efficiency	<code>NOT EXISTS</code> vs <code>LEFT JOIN ... IS NULL</code>	<code>NOT EXISTS</code> uses a fast Hash Anti-Join and stops searching on the first match (early exit); safe with <code>NULL</code> s.
Leftmost Prefix Rule	Index on <code>(branch, date, amount)</code> . Query on <code>WHERE date = ...</code>	Full Table Scan! You cannot skip the leading column of a composite B+ Tree index.
Covering Index	<code>CREATE INDEX ... INCLUDE (amount, status)</code>	Stores payload columns directly in B+ Tree leaf pages without indexing them, enabling fast Index-Only Scans .
COUNT(*) vs COUNT(col)	Table with 10 rows where 3 rows have <code>col = NULL</code>	<code>COUNT(*)</code> returns <code>10</code> . <code>COUNT(col)</code> returns <code>7</code> .
SUM on Empty Table	<code>SELECT SUM(amount) FROM txns WHERE 1=0</code>	Returns <code>NULL</code> , not <code>0</code> ! Always write <code>COALESCE(SUM(amount), 0)</code> in banking queries.



CHAPTER 06 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of 10–Minute Rapid Revision: Tricky JS, Python & SQL Gotchas
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 07

Active Recall Flashcards for IDFC FIRST Bank Interview

PRIORITY

 High Priority

DIFFICULTY

High-Yield Summary

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Active Recall Flashcards for IDFC FIRST Bank Interview

Banking & FinTech Flashcards

Flashcard B1

- **Q:** What is the core rule of double-entry bookkeeping?
- **A:** Every transaction must record at least one Debit and one Credit such that: $\Sigma \text{Debits} = \Sigma \text{Credits}$
The net financial sum of any balanced transaction entry is strictly zero.

Flashcard B2

- **Q:** What is an Idempotency Key in a payment API?
- **A:** A unique client-generated token (typically UUIDv4) transmitted in headers (**X-Idempotency-Key**).
It guarantees that duplicate retries of an identical request will execute the underlying business logic and financial debit exactly once, returning the cached outcome on subsequent calls.

Flashcard B3

- **Q:** What are the 4 main payment rails in India?
- **A:**
 - **UPI:** Instant real-time gross settlement 24/7/365, VPA-based, capped at ₹1–5 Lakh.
 - **IMPS:** Instant 24/7 account-based settlement, capped at ₹5 Lakh.
 - **NEFT:** Deferred Net Settlement in 30-minute batches 24/7, no upper limit.
 - **RTGS:** Real-Time Gross Settlement for high-value transactions (minimum ₹2 Lakh).

Flashcard B4

- **Q:** What does PCI-DSS strictly forbid regarding card security?
- **A:** Never store Sensitive Authentication Data (SAD) after authorization—specifically the 3-digit **CVV/CVC** or raw card PIN, even if encrypted.

SQL & DBMS Flashcards

Flashcard S1

- **Q:** What does ACID stand for?
- **A:**
- **Atomicity:** All statements commit or all rollback (WAL undo logs).
- **Consistency:** All integrity constraints and invariants remain valid.
- **Isolation:** Concurrent transactions execute without mutual interference (2PL/MVCC).
- **Durability:** Committed transactions survive system crashes (WAL `fsync`).

Flashcard S2

- **Q:** What is the difference between `RANK()` and `DENSE_RANK()` ?
- **A:** In ties:
- `RANK()` assigns identical ranks and skips subsequent values (1, 2, 2, 4).
- `DENSE_RANK()` assigns identical ranks and does NOT skip subsequent values (1, 2, 2, 3).

Flashcard S3

- **Q:** What is a Write-Ahead Log (WAL)?
- **A:** An append-only disk log where transaction changes must be recorded and flushed via sequential `fsync()` *before* any actual data pages are modified on disk storage.

Flashcard S4

- **Q:** Why is `NOT IN` dangerous when subquery results contain `NULL` ?
- **A:** Due to SQL's three-valued logic, `x ≠ NULL` evaluates to `UNKNOWN`. If any subquery row is `NULL`, the entire `NOT IN` conjunction evaluates to `UNKNOWN`, causing the query to return zero rows silently. Use `NOT EXISTS` instead.

Node.js & JavaScript Flashcards

Flashcard N1

- **Q:** What is the execution difference between `process.nextTick()` and `setImmediate()` ?
- **A:**
- `process.nextTick()` belongs to the process microtask queue; it runs immediately when the active call stack clears, before the event loop advances.
- `setImmediate()` belongs to the libuv **Check phase** (macrotask). Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)`.

Flashcard N2

- **Q:** Which Node.js asynchronous operations use the 4-thread worker pool?
- **A:** Only `fs`, `crypto`, `zlib`, and `dns.lookup`. Network I/O (`http`, `https`, `tls`, `net`) uses non-blocking OS kernel notifications (`epoll` / `kqueue`) and bypasses the thread pool entirely.

Flashcard N3

- **Q:** What causes a Stale Closure in React?
- **A:** When a hook closure (like `useEffect` or `useCallback`) captures state or prop values from an earlier render because those values were omitted from the dependency array.

System Design Flashcards

Flashcard D1

- **Q:** What is the CAP theorem trade-off in banking?
- **A:** Banking systems are **CP systems** (Consistency & Partition Tolerance). In a network partition, the system rejects writes or degrades availability to guarantee that financial balances are never duplicated or double-spent.

Flashcard D2

- **Q:** How do you guarantee message ordering in Apache Kafka?
- **A:** Kafka guarantees strict message ordering **only within a single partition**. By setting the message key to `account_id`, all events for that account are routed to the same partition and processed strictly in chronological order.

Python & Data Ecosystem Flashcards

Flashcard P1 (Python: Mutability)

- **Q:** List vs Tuple?
- **A:** List mutable hoti hai (can modify in-place via `append/pop`); Tuple immutable hota hai (fixed-size in memory, faster, hashable, can be dictionary keys).

Flashcard P2 (Python: Copying)

- **Q:** Shallow copy vs Deep copy?
- **A:**
- `shallow = arr.copy()`: Outer container naya banta hai, par nested elements ke pointers share hote hain.
- `deep = copy.deepcopy(arr)`: Recursively saare nested objects ki completely independent duplicate copy banti hai.

Flashcard P3 (Python: Generators)

- **Q:** What is a Generator and why use `yield` ?
 - **A:** Generator lazy evaluation use karke stream produce karta hai. Ek time par sirf 1 item memory mein rehta hai, reducing memory from $O(N)$ to $O(1)$.
-

Flashcard NP1 (NumPy: Core Difference)

- **Q:** Why use NumPy ndarray instead of Python list?
 - **A:** NumPy data ko C-contiguous raw memory block mein store karta hai aur CPU SIMD instructions use karke vectorized calculations karta hai bina Python loop overhead ke (40x–80x faster).
-

Flashcard NP2 (NumPy: Broadcasting)

- **Q:** What is Broadcasting in NumPy?
 - **A:** NumPy ka mechanism jisme compatible shapes ke arrays par arithmetic operations allow kiye jaate hain without manually duplicating data in memory. Rule: Trailing dimensions must be equal or one of them must be 1.
-

Flashcard NP3 (NumPy: Views vs Copies)

- **Q:** Does slicing a NumPy array create a copy?
 - **A:** NO! Slicing (`arr[1:4]`) returns a **View** (window to original memory). Modifying the slice mutates the original array! Use `arr[1:4].copy()` for a safe independent copy.
-

Flashcard PD1 (pandas: Indexing)

- **Q:** Difference between `loc` and `iloc` ?
 - **A:**
 - `df.loc[]` : **Label-based** selection (row/column names, end boundary is **inclusive**).
 - `df.iloc[]` : **Integer-position based** selection (0-indexed integer offsets, end boundary is **exclusive**).
-

Flashcard PD2 (pandas: GroupBy)

- **Q:** What is Split-Apply-Combine?
 - **A:** `df.groupby()` ka 3-step engine:
 - **Split:** Data ko keys ke basis par groups mein baanta jata hai.
 - **Apply:** Har group par aggregation (`sum` , `mean` , `count`) compute hoti hai.
 - **Combine:** Results ko ek consolidated summary DataFrame mein merge kiya jata hai.
-

Flashcard PD3 (pandas: Performance)

- **Q:** Why should you avoid `for index, row in df.iterrows():` ?

- **A:** `iterrows()` har row par Python object overhead aur type conversions karta hai, rendering 100x slower. Hamesha vectorized operations ya `np.select` use karein.

Flashcard MP1 (Matplotlib: Anatomy)

- **Q:** Difference between Figure and Axes in Matplotlib?
- **A:**
- **Figure:** Pura outer window/canvas.
- **Axes:** Actual graph ya plotting area (jisme x/y axis, title, ticks, bars, lines hote hain). Ek Figure mein multiple Axes (subplots) ho sakte hain!

CHAPTER 07 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Q: What is the core rule of double-entry bookkeeping?
- A: Every transaction must record at least one Debit and one Credit such that:
- Q: What is an Idempotency Key in a payment API?

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases